

# Density Contracts and Dense Adapters Specification

## LD0-R1 backend-neutral records, provenance, public node access, and compatibility preservation

mdstats

2026-07-20

### Contents

|                                                           |    |
|-----------------------------------------------------------|----|
| Purpose and implementation status .....                   | 2  |
| Authority and relation to the architecture standard ..... | 2  |
| Motivation .....                                          | 2  |
| Scope .....                                               | 3  |
| Included .....                                            | 3  |
| Excluded .....                                            | 3  |
| Dependency direction .....                                | 3  |
| Canonical immutable metadata .....                        | 4  |
| JSON-compatible value domain .....                        | 4  |
| FrozenJSONMapping .....                                   | 4  |
| Source identity .....                                     | 4  |
| Tagged canonical source key .....                         | 4  |
| Persistent provenance .....                               | 4  |
| Unified weighted samples .....                            | 5  |
| Input constraints .....                                   | 5  |
| Shared option records .....                               | 5  |
| Resolution .....                                          | 5  |
| Kernel .....                                              | 5  |
| Storage .....                                             | 6  |
| Rendering .....                                           | 6  |
| Compatibility wrappers .....                              | 6  |
| Operational compatibility matrix .....                    | 7  |
| Backend-neutral field identity .....                      | 7  |
| Public periodic node access .....                         | 7  |
| Dense iteration order .....                               | 8  |
| Periodic gather .....                                     | 8  |
| Dense field adaptation .....                              | 8  |
| Storage summary .....                                     | 8  |
| Zero-copy adapter .....                                   | 8  |
| Serialization .....                                       | 8  |
| Scene and framework integration .....                     | 9  |
| Error semantics .....                                     | 9  |
| Edge cases .....                                          | 9  |
| Empty provenance dimensions .....                         | 9  |
| Zero density nodes .....                                  | 9  |
| Negative and aliased gather indices .....                 | 9  |
| Nested metadata arrays .....                              | 9  |
| Nonfinite metadata .....                                  | 9  |
| Legacy metadata .....                                     | 9  |
| Acceptance tests .....                                    | 9  |
| Focused test inventory .....                              | 10 |

|                                          |    |
|------------------------------------------|----|
| Completion condition and next gate ..... | 10 |
| References .....                         | 10 |

## Purpose and implementation status

This document is the normative stage specification for **LD0-R1** of the `mdstats` dynamical-framework and density plotting architecture.

Package version:

0.19.40a0

Implementation modules:

```
mdstats/plotting/density_contracts.py
mdstats/plotting/atomic_density.py
mdstats/plotting/framework_density.py
mdstats/plotting/framework_dynamics.py
```

LD0-R1 is implemented. It establishes backend-neutral contracts and adapts the existing dense density field to them. It does **not** change the scientific estimator, registration, grid selection, FFT smoothing, normalization, highest-density thresholds, mesh extraction, or Plotly composition.

The current operational path remains

registered samples → periodic trilinear CIC deposition → `legacy_spectral_v1` → normalized dense field .

The CIC particle-mesh assignment is the existing implementation attributed to Hockney and Eastwood. LD0-R1 does not introduce or adapt a new numerical algorithm; its new behavior is project-specific API, validation, immutability, provenance, and serialization design.

## Authority and relation to the architecture standard

The single governing plan is:

`docs/arch_manuals/mdstats_dynamical_framework_density_architecture_standard.md`

This stage specification refines only the LD0-R1 implementation boundary. If it conflicts with the architecture standard, the architecture standard governs.

The former standalone local-sparse roadmap is not an implementation authority.

## Motivation

The dense scalar field previously combined four concerns:

1. scientific identity;
2. dense-array storage;
3. renderer-specific assumptions;
4. ad hoc metadata and atom-index provenance.

A sparse backend cannot be added safely while scene and renderer code require one concrete dense class. LD0-R1 introduces a stable read-only boundary before any operator, broadening, registration, or storage migration occurs.

The immediate goals are:

- one source record usable by atomic, vertex, and edge channels;
- structured source identity beyond integer atom indices;
- common resolution, kernel, storage, and render policies;
- backend-neutral field identity and node access;
- exact preservation of the current dense numerical path;
- explicit rejection of future identifiers whose implementation gates are not complete.

## Scope

### Included

LD0-R1 provides:

- FrozenJSONMapping;
- canonical tagged source keys;
- DensitySourceProvenance;
- PeriodicWeightedSamples3D;
- DensityStorageSummary;
- DensityResolutionOptions;
- DensityKernelOptions;
- DensityStorageOptions;
- DensityRenderOptions;
- ScalarField3D;
- PeriodicNodeFieldAccess;
- DensePeriodicNodeFieldAdapter;
- dense-field storage summaries;
- lexicographic dense-node iteration;
- periodic dense-node gather;
- schema-versioned JSON-compatible round trips;
- compatibility normalization inside existing atomic/framework option classes;
- backend-neutral scene acceptance of scalar fields;
- reserved identifier validation;
- package-root and mdstats.plotting exports.

### Excluded

LD0-R1 does not provide:

- the canonical finite-support discrete Gaussian operator;
- CIC-plus-stencil effective broadening;
- variable-cell laboratory-density rejection;
- periodic-mean convergence diagnostics;
- node-cloud coordinate correction;
- global Phase-A/Phase-B scene planning;
- sparse CIC aggregation;
- sparse block storage;
- sparse HDR or mesh extraction;
- automatic dense/sparse selection.

These belong to LD0-R2, LD0-R3, LD0-K, LD0-B, and LD1-LD4.

## Dependency direction

The new module is renderer-independent:

```
density_contracts
  -> graph_errors
  -> NumPy / Python standard library
```

It must not import:

```
Plotly
scikit-image
framework_dynamics
atomic_density
framework_density
```

The existing scientific modules import the contracts, not the reverse.

## Canonical immutable metadata

### JSON-compatible value domain

Metadata may contain only:

- strings;
- finite integers and floats;
- booleans;
- None;
- string-keyed mappings;
- nested sequences of allowed values;
- NumPy scalar values reducible to the above;
- NumPy arrays reducible to nested sequences.

Nonfinite floats and arbitrary Python objects are rejected.

### FrozenJSONMapping

```
class FrozenJSONMapping(Mapping[str, CanonicalJSONValue]):  
    def to_json_dict(self) -> dict[str, Any]: ...  
    def canonical_json(self) -> str: ...
```

Normative behavior:

- keys are strings and iterate in lexical order;
- mappings are recursively frozen;
- sequences are stored as tuples;
- output sequences become JSON arrays;
- output uses sorted keys and disallows NaN/infinity;
- mutation through any nesting level is impossible.

The canonical emitter is a project-specific deterministic JSON representation. It does not claim complete conformance to an external JSON canonicalization scheme.

## Source identity

### Tagged canonical source key

A CanonicalSourceKey is a nonempty recursively JSON-compatible tuple whose first member is a nonempty string tag.

Examples:

```
("vertex", "T", 4, ("shift", 0, 1, -1))  
("edge", ("node", 2), ("node", 9), ("shift", 1, 0, 0), 3)
```

Keys are normalized and sorted by their deterministic JSON representation. Arbitrary Python hashables are not serialized directly.

### Persistent provenance

```
@dataclass(frozen=True, slots=True)  
class DensitySourceProvenance:  
    schema_version: str  
    source_kind: str  
    atom_indices: tuple[int, ...] = ()  
    vertex_keys: tuple[CanonicalSourceKey, ...] = ()  
    edge_keys: tuple[CanonicalSourceKey, ...] = ()  
    metadata: FrozenJSONMapping = FrozenJSONMapping()
```

Constraints:

- `source_kind` is nonempty;
- atom indices are nonnegative, sorted, and unique;
- keys are tagged and recursively JSON-compatible;
- metadata are recursively frozen;
- the schema version must be supported.

The current dense atomic and framework fields populate `source_kind` and `atom_indices`. Complete projected-vertex and multiedge keys are added at the source-preparation boundary in a later gate without changing this schema.

## Unified weighted samples

```
@dataclass(frozen=True, slots=True)
class PeriodicWeightedSamples3D:
    fractional_positions: NDArray[np.float64]
    weights: NDArray[np.float64]
    source_provenance: DensitySourceProvenance
    total_measure: float
    measure_kind: Literal["occupancy", "arc_length"]
    measure_units: str
    sample_group_ids: NDArray[np.int64] | None = None
    metadata: FrozenJSONMapping = FrozenJSONMapping()
    schema_version: str = ...
```

### Input constraints

**`fractional_positions`** shape  $(n\_samples, 3)$ , finite, C-contiguous, and folded into  $[0,1)$ .

**`weights`** shape  $(n\_samples,)$ , finite, nonnegative, and

$$\left| \sum_s w_s - M \right| \leq 5 \times 10^{-13} \max(1, M),$$

where  $M = \text{total\_measure}$ .

**`sample_group_ids`** optional shape  $(n\_samples,)$ , integer, nonnegative transient parent mapping.

**`measure_kind`** occupancy or arc\_length.

All arrays are defensive read-only copies. Transient group IDs are not copied into final field provenance.

## Shared option records

### Resolution

```
DensityResolutionOptions(
    grid_shape=None,
    grid_interval=0.20,
    gaussian_bandwidth=None,
    gaussian_to_grid_ratio=2.0,
    adaptive_smearing=True,
    max_smearing_to_sample_sd_ratio=0.50,
    sample_sd_quantile=0.10,
    broadening_metric="gaussian_sigma_v1",
)
```

The record owns scientific resolution and broadening-policy identity. LD0-R1 supports only `gaussian_sigma_v1` operationally.

### Kernel

```
DensityKernelOptions(
    smoothing_operator="legacy_spectral_v1",
    kernel_tail_tolerance=1.0e-8,
)
```

The tail tolerance is validated in

$$10^{-15} \leq \varepsilon \leq 10^{-3}.$$

It is reserved metadata for the legacy operator and becomes operational for `discrete_periodized_v1` in LD0-K.

## Storage

```
DensityStorageOptions(  
    grid_backend="dense",  
    local_block_shape=(16, 16, 16),  
    sparse_activation_fraction=0.20,  
)
```

Only `dense` is operational. `local_sparse` and `auto` are recognized identifiers but rejected when attached to an active atomic/framework density request.

## Rendering

```
DensityRenderOptions(  
    mass_fractions=(0.50, 0.80, 0.95),  
    render_mode="mesh",  
    display_replication="canonical",  
    standalone_final_mesh_faces=250_000,  
    cloud_max_points=40_000,  
)
```

`standalone_final_mesh_faces` is a terminal limit only for mesh preparation that has no scene fitting controller. `max_mesh_faces` remains a deprecated constructor, attribute, and serialized alias during migration. Scene-assigned visual targets and runtime-derived raw extraction limits are represented by `DensityMeshFaceContract`, not by `DensityRenderOptions`.

The existing atomic and framework render option classes retain compatibility with old constructors and expose a normalized `render_options` record.

## Compatibility wrappers

`AtomicDensityOptions` and `FrameworkDensityOptions` retain all existing direct fields. They additionally accept:

```
resolution_options=...  
kernel_options=...  
storage_options=...
```

When `resolution_options` is supplied, it is authoritative and its values are copied to the legacy compatibility properties used by current numerical code. When omitted, one normalized record is constructed from the legacy fields.

This preserves existing code such as:

```
AtomicDensityOptions(grid_shape=(16, 16, 16), gaussian_bandwidth=0.25)
```

while allowing the target composition:

```
AtomicDensityOptions(  
    resolution_options=DensityResolutionOptions(...),  
    kernel_options=DensityKernelOptions(...),  
    storage_options=DensityStorageOptions(...),  
)
```

## Operational compatibility matrix

| Backend      | legacy_spectral_v1 | discrete_periodized_v1 |
|--------------|--------------------|------------------------|
| dense        | implemented        | rejected until LD0-K   |
| local_sparse | rejected           | rejected until LD1-B   |
| auto         | rejected until LD4 | rejected until LD4     |

Broadening:

| Metric                       | Status               |
|------------------------------|----------------------|
| gaussian_sigma_v1            | implemented          |
| effective_cic_stencil_rms_v1 | rejected until LD0-B |

Unsupported combinations raise `GraphUnsupportedFeatureError`. They are never accepted and silently mapped to the dense legacy path.

## Backend-neutral field identity

```
@runtime_checkable
class ScalarField3D(Protocol):
    schema_version: str
    field_key: str
    label: str
    physical_units: str
    display_cell: NDArray[np.float64]
    total_measure: float
    gaussian_bandwidth: float
    smoothing_operator: str
    broadening_metric: str
    storage_backend: str
    source_provenance: DensitySourceProvenance
    metadata: FrozenJSONMapping

    @property
    def grid_shape(self) -> tuple[int, int, int]: ...
    @property
    def voxel_volume(self) -> float: ...
    @property
    def integral(self) -> float: ...
    def threshold_for_mass_fraction(self, q: float) -> float: ...
    def storage_summary(self) -> DensityStorageSummary: ...
```

The protocol describes scientific identity and storage accounting, not concrete array layout.

## Public periodic node access

```
@runtime_checkable
class PeriodicNodeFieldAccess(Protocol):
    def iter_stored_nodes(
        self,
        *,
        batch_size: int | None = None,
    ) -> Iterator[
        tuple[NDArray[np.int64], NDArray[np.float64]]
    ]: ...

    def gather_node_values(
        self,
```

```

        logical_indices: NumpyArray[np.int64],
    ) -> NumpyArray[np.float64]: ...

```

## Dense iteration order

For shape (N1,N2,N3), dense iteration is exact lexicographic C order:

(0,0,0), (0,0,1), ..., (0,1,0), ..., (N1-1,N2-1,N3-1)

Batches and values are read-only. `batch_size=None` emits one batch.

## Periodic gather

Input has shape (n,3) and integer dtype. Each axis is reduced modulo the logical shape. Returned values are float64, shape (n,), and read-only.

Sparse missing-node semantics are reserved: absent nodes return zero after LD1-B.

## Dense field adaptation

PeriodicScalarField3D directly implements both protocols and retains:

- the values dense array;
- `grid_shape` and `voxel_volume` properties;
- `integral` as a property;
- current HDR threshold behavior;
- `selected_atom_indices` for compatibility;
- optional sample positions.

New properties derive from structured metadata:

```

schema_version
physical_units
smoothing_operator
broadening_metric
storage_backend = dense
source_provenance

```

## Storage summary

For dense fields:

$$N_{\text{logical}} = N_{\text{stored}} = N_1 N_2 N_3,$$

`stored_block_count = 0`, and the scalar storage estimate is the dense array byte count. Nonzero count is measured explicitly.

## Zero-copy adapter

DensePeriodicNodeFieldAdapter.`from_field`(field)

The adapter stores the original field reference and the exact original values object. It rejects writable or non-float64 arrays. It does not reconstruct, reshape, or copy scientific values.

## Serialization

The following records support to\_json\_dict() and from\_json\_dict():

- DensitySourceProvenance;
- PeriodicWeightedSamples3D;
- DensityStorageSummary;
- all four shared option records;
- PeriodicScalarField3D.

Dense field serialization includes scientific values only when requested. The default round-trip includes values and reconstructs read-only arrays.

Round trips preserve:



- schema versions;
- every public option field;
- full tagged source keys;
- recursively frozen metadata;
- array values and shapes;
- field scientific identity.

## Scene and framework integration

`FrameworkDensityFields` accepts any object satisfying `ScalarField3D` instead of requiring `PeriodicScalarField3D` by concrete type. It preserves dimensionally separate vertex and edge channels and its `edge_source` contract.

`FrameworkDynamicsScene.atomic_density_fields` similarly accepts `tuple[ScalarField3D, ...]`. Cell compatibility checks continue to use public `display_cell` values.

Current render helpers still receive dense fields during LD0-R1. Sparse renderer dispatch is introduced only after its owning gate.

## Error semantics

**GraphStyleError** malformed shapes, intervals, bandwidths, fractions, block shapes, or tail tolerance.

**GraphAdapterError** malformed arrays, metadata, source keys, provenance, schema versions, normalization, or node indices.

**GraphUnsupportedFeatureError** recognized operator, broadening, or backend identifier whose implementation gate is incomplete.

**TypeError** wrong record or protocol object type.

## Edge cases

### Empty provenance dimensions

A source may have no atom indices when complete vertex or edge keys identify it. LD0-R1 dense compatibility fields continue to carry nonempty atom indices.

### Zero density nodes

Dense iteration includes stored zeros because all dense logical nodes are allocated. The storage summary distinguishes nonzero nodes from stored values.

### Negative and aliased gather indices

Periodic gather accepts negative and out-of-range integer indices and applies modulo indexing. Floating-point indices are rejected.

### Nested metadata arrays

NumPy arrays in metadata are converted to immutable nested tuples. Scientific arrays belong in explicit record fields rather than metadata when their shape or dtype is part of the contract.

### Nonfinite metadata

NaN and infinity are rejected because they do not have portable JSON semantics.

### Legacy metadata

Fields lacking explicit `smoothing_operator` or `broadening_metric` metadata expose `legacy_spectral_v1` and `gaussian_sigma_v1` respectively. Serialization does not mutate or expand the original metadata mapping.

## Acceptance tests

LD0-R1 passes only if:

1. all existing atomic-density, framework-density, and framework-dynamics focused tests pass;
2. dense scientific arrays are numerically identical to `0.19.39a0` for matched fixtures;
3. every public scientific array is read-only;

4. nested metadata are recursively immutable;
5. provenance and all option records survive JSON-compatible round trips;
6. dense field values survive exact round trip;
7. dense adapter values are the same object and share memory with the field;
8. dense node iteration is lexicographic and complete;
9. gather implements periodic modulo semantics;
10. unimplemented identifiers fail explicitly;
11. `density_contracts.py` has no Plotly import.

## Focused test inventory

`tests/test_density_contracts.py`  
`tests/test_atomic_density.py`  
`tests/test_framework_density.py`  
`tests/test_framework_dynamics.py`

The new contract suite covers:

- option normalization;
- reserved identifier rejection;
- tagged provenance ordering and round trip;
- weighted-sample validation;
- runtime protocol satisfaction;
- dense node iteration and gather;
- zero-copy adaptation;
- storage-summary round trip;
- dense-field round trip;
- recursive immutability;
- dependency hygiene.

## Completion condition and next gate

LD0-R1 is complete when the implementation and focused tests satisfy this specification without changing dense scientific output.

The next gate is **LD0-R2**, which owns:

- cell-equivalence validation;
- periodic-mean diagnostics;
- quantile and zero-spread policy;
- reciprocal-resolution diagnostics;
- variable-cell laboratory-density restriction;
- logical-node cloud-coordinate correction.

No LD0-K, LD0-B, or sparse numerical feature is implied by completing LD0-R1.

## References

1. Hockney, R. W., and J. W. Eastwood. *Computer Simulation Using Particles*. Bristol: Adam Hilger, 1988. Existing cloud-in-cell attribution; no new particle-mesh algorithm is introduced in LD0-R1.